Programming Lab 9 - Campus Bookstore Inventory System

Overview

The campus bookstore needs a simple inventory system to keep track of products, product information, and product categories. The owner wants to quickly locate products by Product ID, maintain accurate inventory information, and prevent duplicate categories from being stored.

This lab builds upon previous concepts including variables, branching, loops, functions, strings, and lists, and introduces dictionaries, tuples, sets, and dictionary comprehensions. The purpose of this lab is to select the most appropriate collection type for each programming task.

Reminder

Programming labs often require multiple attempts and revisions. Begin by completing the Program Planning section, then implement and test your solution.

Part 1 – Program Planning

Professional software developers rarely begin writing code immediately. Before implementation begins, team members work together to review the project requirements, discuss possible solutions, and make design decisions about how the program will be organized.

Before beginning this programming assignment, participate in the **Programming Lab 9: Design Review**. During the discussion, you will work with your classmates to evaluate different design ideas for the Campus Bookstore Inventory System. The goal is to think through the problem, justify your design decisions, and consider alternative approaches **before** writing any code.

After participating in the discussion, use the ideas you developed to design and implement **your own** solution.

Note: Although you will discuss design ideas with your classmates, all submitted work must be your own.

Part 2 – Program Implementation

Business Rules

- Each product has a unique Product ID.
- Each product belongs to one category.
- Each product stores a product name, category, price, and quantity.
- Categories should not contain duplicates.
- Products with a quantity less than 5 are considered low stock.

Program Requirements

Your program must display a ***menu*** that allows the user to perform the following tasks: **(See Appendix for additional details and a sample program run)**

1. Add a new product to the inventory.

2. Display all products in the inventory.
3. Search for a product using its Product ID.
4. Display Inventory Report
5. Exit the program.

Collection Requirements (See Appendix for additional details)

- Use a dictionary to store products using the Product ID as the key.
- Use a tuple to store the product name, category, price, and quantity as the dictionary value.
- Use a set to store product categories.
- Use a list to store products needing to be reordered.

Program Must Include

- Functions
- At least one while loop
- Input validation
- Meaningful comments
- Formatted output
- Appropriate use of a dictionary, tuple, set, and list

Challenge Extensions (Optional)

- Update an existing product.
- Delete a product.
- Sort products alphabetically.
- Allow products to belong to multiple categories.

Part 3 – Program Testing

After completing your program, verify that it works correctly by testing all required functionality.

Submit **at least five test cases**.

- Test all menu options.
- Verify that products can be added correctly.
- Verify searching by Product ID.
- Verify duplicate categories are not stored.
- Verify low-stock products are displayed correctly.
- Verify the summary information is correct.

Submission Requirements

- Source code
- Test cases
- Code walk-through video (3–4 minutes) demonstrating program runs

Appendix

Menu

1. Add Product
Prompt the user to enter the Product ID, product name, category, price, and quantity. Store the information using the required collections.

2. Display All Products
Display every product currently stored in the inventory.

3. Search for a Product
Allow the user to search for a product using its Product ID. If the Product ID is not found, display an appropriate message.

4. Display Inventory Report
The Inventory Summary displays:

- Total number of products in inventory
- All product categories currently in the inventory.
- Low Stock: all products with a quantity less than 5.
- Total inventory value

Sample Program Run

Campus Bookstore Inventory System

1. Add Product
2. Display All Products
3. Search for a Product
4. Display Inventory Report
5. Exit

Enter your choice: 1

Enter Product ID: B101
Enter Product Name: Python Programming
Enter Category: Books
Enter Price: 79.95
Enter Quantity: 12

Product added successfully.

Collection Requirements

Your program must demonstrate the appropriate use of **each** of the following Python collections.

Dictionary

Use a **dictionary** to store the bookstore inventory.

- The key must be the Product ID (for example, "B101" or "P205").
- The value must contain the product information as a tuple.
- Each Product ID should be unique.

Example:
inventory["B101"] = ("Python Programming", "Books", 79.95, 12)

Tuple

Use a **tuple** to store the information for each product.
Each tuple must contain:

- Product Name
- Category
- Product Price
- Quantity on Hand

Because these values describe a single product, they should be stored as a single tuple.

Example:
("Python Programming", "Books", 79.95, 12)

Set

Use a **set** to store all product categories.

Examples of categories include:

- Books
- Electronics
- School Supplies
- Clothing

After each product is added, the program should add the category to the **set**. Because sets automatically eliminate duplicate values, each category should appear only once.

Example:
{"Books", "Electronics", "Clothing"}

List

Create a **list** of Product IDs (or product names) for all products that need to be reordered. A product should be considered low stock if its quantity is less than 5.

Create the list by examining the inventory dictionary. As the program loops through each product, add the Product ID to the list if the quantity on hand is less than 5.

Example:
["B101", "P205", "S310"]